



# Cyberscope

A *TAC Security* Company

## Audit Report

# Gravity Protocol Token

March 2026

SHA256

o24ab7741da55d2ccc91a41bbdc5dd9278aa165ab5004ed6f6fcd7f6a1d4d6ac

Audited by © cyberscope

# Analysis

● Critical   ● Medium   ● Minor / Informative   ● Pass

| Severity | Code | Description             | Status     |
|----------|------|-------------------------|------------|
| ●        | ST   | Stops Transactions      | Unresolved |
| ●        | OTUT | Transfers User's Tokens | Passed     |
| ●        | ELFM | Exceeds Fees Limit      | Unresolved |
| ●        | MT   | Mints Tokens            | Passed     |
| ●        | BT   | Burns Tokens            | Passed     |
| ●        | BC   | Blacklists Addresses    | Unresolved |

# Diagnostics

● Critical ● Medium ● Minor / Informative

| Severity | Code | Description                              | Status       |
|----------|------|------------------------------------------|--------------|
| ●        | CCR  | Contract Centralization Risk             | Unresolved   |
| ●        | IDI  | Immutable Declaration Improvement        | Unresolved   |
| ●        | IMWL | Incorrect Max Wallet Limitation          | Acknowledged |
| ●        | IC   | Inefficient Check                        | Unresolved   |
| ●        | MEE  | Missing Events Emission                  | Unresolved   |
| ●        | PLPI | Potential Liquidity Provision Inadequacy | Unresolved   |
| ●        | PTRP | Potential Transfer Revert Propagation    | Unresolved   |
| ●        | SMS  | Swap Mechanism Suppression               | Unresolved   |
| ●        | L13  | Divide before Multiply Operation         | Unresolved   |
| ●        | L16  | Validate Variable Setters                | Unresolved   |

# Table of Contents

|                                                 |          |
|-------------------------------------------------|----------|
| <b>Analysis</b>                                 | <b>1</b> |
| <b>Diagnostics</b>                              | <b>2</b> |
| <b>Table of Contents</b>                        | <b>3</b> |
| <b>Risk Classification</b>                      | <b>5</b> |
| <b>Review</b>                                   | <b>6</b> |
| Audit Updates                                   | 6        |
| Source Files                                    | 6        |
| <b>Findings Breakdown</b>                       | <b>7</b> |
| ST - Stops Transactions                         | 8        |
| Description                                     | 8        |
| Recommendation                                  | 8        |
| BC - Blacklists Addresses                       | 10       |
| Description                                     | 10       |
| Recommendation                                  | 11       |
| ELFM - Exceeds Fees Limit                       | 12       |
| Description                                     | 12       |
| Recommendation                                  | 13       |
| CCR - Contract Centralization Risk              | 14       |
| Description                                     | 14       |
| Recommendation                                  | 15       |
| IDI - Immutable Declaration Improvement         | 16       |
| Description                                     | 16       |
| Recommendation                                  | 16       |
| IMWL - Incorrect Max Wallet Limitation          | 17       |
| Description                                     | 17       |
| Recommendation                                  | 17       |
| IC - Inefficient Check                          | 18       |
| Description                                     | 18       |
| Recommendation                                  | 18       |
| MEE - Missing Events Emission                   | 19       |
| Description                                     | 19       |
| Recommendation                                  | 19       |
| PLPI - Potential Liquidity Provision Inadequacy | 20       |
| Description                                     | 20       |
| Recommendation                                  | 21       |
| PTRP - Potential Transfer Revert Propagation    | 22       |
| Description                                     | 22       |
| Recommendation                                  | 23       |
| SMS - Swap Mechanism Suppression                | 24       |

|                                        |           |
|----------------------------------------|-----------|
| Description                            | 24        |
| Recommendation                         | 25        |
| L13 - Divide before Multiply Operation | 26        |
| Description                            | 26        |
| Recommendation                         | 26        |
| L16 - Validate Variable Setters        | 27        |
| Description                            | 27        |
| Recommendation                         | 27        |
| <b>Functions Analysis</b>              | <b>28</b> |
| <b>Inheritance Graph</b>               | <b>31</b> |
| <b>Flow Graph</b>                      | <b>32</b> |
| <b>Summary</b>                         | <b>33</b> |
| <b>Disclaimer</b>                      | <b>34</b> |
| <b>About Cyberscope</b>                | <b>35</b> |

# Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation:** This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation:** This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical:** Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium:** Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor:** Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative:** Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

| Severity              | Likelihood / Impact of Exploitation                      |
|-----------------------|----------------------------------------------------------|
| ● Critical            | Highly Likely / High Impact                              |
| ● Medium              | Less Likely / High Impact or Highly Likely/ Lower Impact |
| ● Minor / Informative | Unlikely / Low to no Impact                              |

# Review

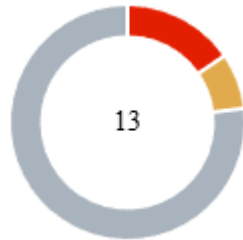
## Audit Updates

|                          |                                                                                                                                                                              |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Initial Audit</b>     | 11 Mar 2026<br><br><a href="https://github.com/cyberscope-io/audits/blob/main/1-grav/v1/audit.pdf">https://github.com/cyberscope-io/audits/blob/main/1-grav/v1/audit.pdf</a> |
| <b>Corrected Phase 2</b> | 25 Mar 2026                                                                                                                                                                  |

## Source Files

| Filename                                 | SHA256                                                               |
|------------------------------------------|----------------------------------------------------------------------|
| <b>GravityProtocol_v4_post_audit.sol</b> | a24ab7741da55d2ccc91a41bbdc5dd927<br>8aa165ab5004ed6f6fcd7f6a1d4d6ac |

## Findings Breakdown



|                       |    |
|-----------------------|----|
| ● Critical            | 2  |
| ● Medium              | 1  |
| ● Minor / Informative | 10 |

| Severity              | Unresolved | Acknowledged | Resolved | Other |
|-----------------------|------------|--------------|----------|-------|
| ● Critical            | 2          | 0            | 0        | 0     |
| ● Medium              | 1          | 0            | 0        | 0     |
| ● Minor / Informative | 9          | 1            | 0        | 0     |



## ST - Stops Transactions

|                    |                                            |
|--------------------|--------------------------------------------|
| <b>Criticality</b> | Critical                                   |
| <b>Location</b>    | GravityProtocol_v4_post_audit.sol#L444,576 |
| <b>Status</b>      | Unresolved                                 |

### Description

Trading is initially disabled for all users excluding the owner. The owner can enable trading for all users and once enabled, cannot disable it again.

Additionally, transactions can be stopped as described in the `PTRP` finding.

Shell

```
if (from != owner() && to != owner()) {  
    require(tradingEnabled, "Trading not yet enabled");  
}
```

### Recommendation

The team should follow the recommendations of the `PTRP` finding. Additionally, the team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

#### Temporary Solutions:

These measurements do not decrease the severity of the finding

- Introduce a time-locker mechanism with a reasonable delay.

- Introduce a multi-signature wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

Permanent Solution:

- Renouncing the ownership, which will eliminate the threats but it is non-reversible.

## BC - Blacklists Addresses

|                    |                                            |
|--------------------|--------------------------------------------|
| <b>Criticality</b> | Critical                                   |
| <b>Location</b>    | GravityProtocol_v4_post_audit.sol#L361,441 |
| <b>Status</b>      | Unresolved                                 |

### Description

The contract owner has the authority to stop addresses from transactions. The owner may take advantage of it by calling the `setBlacklist` function. While the blacklist functionality is limited to a 48-hour window after launch, this restriction also prevents the removal of blacklisted addresses after the deadline expires, making any blacklist entry permanent and irreversible.

Shell

```
function setBlacklist(address account, bool value)
external onlyOwner {
    require(block.timestamp <= blacklistDeadline,
        "Blacklist period expired (48h after launch)");
    isBlacklisted[account] = value;
    emit BlacklistUpdated(account, value);
}

function _transfer(address from, address to, uint256
amount) private {
    ...
    require(!isBlacklisted[from] && !isBlacklisted[to],
        "Blacklisted address");
    ...
}
```

## Recommendation

The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

### Temporary Solutions:

These measurements do not decrease the severity of the finding

- Introduce a time-locker mechanism with a reasonable delay.
- Introduce a multi-signature wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

### Permanent Solution:

- Renouncing the ownership, which will eliminate the threats but it is non-reversible.

## ELFM - Exceeds Fees Limit

|                    |                                                |
|--------------------|------------------------------------------------|
| <b>Criticality</b> | Medium                                         |
| <b>Location</b>    | GravityProtocol_v4_post_audit.sol#L145,147,618 |
| <b>Status</b>      | Unresolved                                     |

### Description

During the anti-snipe period, all buyers face a 99% tax applied in `_tokenTransferSnipe()`. The owner may extend this period by calling the `setAntiBotBlocks()` function with a higher value, up to 20 blocks.

Shell

```
uint256 public antiBotBlocks = 10;

uint256 public constant SNIPE_TAX_RATE = 99;

function _tokenTransferSnipe(address sender, address
recipient, uint256 tAmount) private {
    ...
    uint256 taxAmount = (tAmount * SNIPE_TAX_RATE) / 100;
    ...
}
```

## Recommendation

After the first 20 block pass the fees can be up to 25% which is the maximum accepted fee value. The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

### Temporary Solutions:

These measurements do not decrease the severity of the finding

- Introduce a time-locker mechanism with a reasonable delay.
- Introduce a multi-signature wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

### Permanent Solution:

- Renouncing the ownership, which will eliminate the threats but it is non-reversible.

## CCR - Contract Centralization Risk

|                    |                                                                                                |
|--------------------|------------------------------------------------------------------------------------------------|
| <b>Criticality</b> | Minor / Informative                                                                            |
| <b>Location</b>    | GravityProtocol_v4_post_audit.sol#L317,327,341,346,353,361,368,379,385,391,397,402,409,415,421 |
| <b>Status</b>      | Unresolved                                                                                     |

### Description

The contract's functionality and behavior are heavily dependent on external parameters or configurations. While external configuration can offer flexibility, it also poses several centralization risks that warrant attention. Centralization risks arising from the dependence on external configuration include Single Point of Control, Vulnerability to Attacks, Operational Delays, Trust Dependencies, and Decentralization Erosion.

Shell

```
function excludeFromReward(address account) public
onlyOwner {...}
function includeInReward(address account) external
onlyOwner {...}
function excludeFromFee(address account) public onlyOwner
{...}
function includeInFee(address account) public onlyOwner
{...}
function enableTrading() external onlyOwner {...}
function setBlacklist(address account, bool value) external
onlyOwner {...}
function setProtocolFundWallet(address payable newWallet)
external onlyOwner {...}
function setMaxTxAmount(uint256 amount) external onlyOwner
{...}
function setMaxWalletAmount(uint256 amount) external
onlyOwner {...}
function setSwapAndLiquifyEnabled(bool _enabled) public
onlyOwner {...}
function setCooldownEnabled(bool _enabled) external
```

```
onlyOwner {...}  
function setCooldownTime(uint256 _seconds) external  
onlyOwner {...}  
function setAntiBotBlocks(uint256 _blocks) external  
onlyOwner {...}  
function setEarlySellerExtraFee(uint256 _fee) external  
onlyOwner {...}  
  
function setNumTokensSellToAddToLiquidity(uint256 amount)  
external onlyOwner {...}
```

## Recommendation

To address this finding and mitigate centralization risks, it is recommended to evaluate the feasibility of migrating critical configurations and functionality into the contract's codebase itself. This approach would reduce external dependencies and enhance the contract's self-sufficiency. It is essential to carefully weigh the trade-offs between external configuration flexibility and the risks associated with centralization.



## IDI - Immutable Declaration Improvement

|                    |                                        |
|--------------------|----------------------------------------|
| <b>Criticality</b> | Minor / Informative                    |
| <b>Location</b>    | GravityProtocol_v4_post_audit.sol#L229 |
| <b>Status</b>      | Unresolved                             |

### Description

The contract declares state variables that their value is initialized once in the constructor and are not modified afterwards. The `immutable` is a special declaration for this kind of state variables that saves gas when it is defined.

```
Shell
_routerApproved
```

### Recommendation

By declaring a variable as immutable, the Solidity compiler is able to make certain optimizations. This can reduce the amount of storage and computation required by the contract, and make it more gas-efficient.

## IMWL - Incorrect Max Wallet Limitation

|                    |                                        |
|--------------------|----------------------------------------|
| <b>Criticality</b> | Minor / Informative                    |
| <b>Location</b>    | GravityProtocol_v4_post_audit.sol#L477 |
| <b>Status</b>      | Acknowledged                           |

### Description

The `_transfer` function has a requirement that checks if the balance of the receiver plus the amount sent is less than or equal to the max wallet amount. However the actual amount received will be less due to the fees applied. As a result the restriction is more strict than intended.

Shell

```
require(balanceOf(to) + amount <= maxWalletAmount, "Exceeds  
max wallet amount");
```

### Recommendation

The team is advised to enforce max wallet amount restriction based on the amount transferred to the receiver after the fee calculation.

## IC - Inefficient Check

|                    |                                            |
|--------------------|--------------------------------------------|
| <b>Criticality</b> | Minor / Informative                        |
| <b>Location</b>    | GravityProtocol_v4_post_audit.sol#L209,370 |
| <b>Status</b>      | Unresolved                                 |

### Description

The contract tries to ensure that `protocolFundWallet` is not the deployer or the current owner of the contract; however, an external user may have a different account, essentially making the check inefficient.

```
Shell
require(_protocolFundWallet != _msgSender(), "Protocol fund
wallet must differ from deployer");
...
require(newWallet != owner(), "Protocol fund wallet must
differ from owner");
```

### Recommendation

The team is advised to remove these restrictions as they can easily be bypassed in order to increase gas optimization and efficiency.

## MEE - Missing Events Emission

|                    |                                            |
|--------------------|--------------------------------------------|
| <b>Criticality</b> | Minor / Informative                        |
| <b>Location</b>    | GravityProtocol_v4_post_audit.sol#L317,327 |
| <b>Status</b>      | Unresolved                                 |

### Description

The contract performs actions and state mutations from external methods that do not result in the emission of events. Emitting events for significant actions is important as it allows external parties, such as wallets or dApps, to track and monitor the activity on the contract. Without these events, it may be difficult for external parties to accurately determine the current state of the contract.

Shell

```
function excludeFromReward(address account) public  
    onlyOwner  
  
function includeInReward(address account) external  
    onlyOwner
```

### Recommendation

It is recommended to include events in the code that are triggered each time a significant action is taking place within the contract. These events should include relevant details such as the user's address and the nature of the action taken. By doing so, the contract will be more transparent and easily auditable by external parties. It will also help prevent potential issues or disputes that may arise in the future.

## PLPI - Potential Liquidity Provision Inadequacy

|                    |                                            |
|--------------------|--------------------------------------------|
| <b>Criticality</b> | Minor / Informative                        |
| <b>Location</b>    | GravityProtocol_v4_post_audit.sol#L593,605 |
| <b>Status</b>      | Unresolved                                 |

### Description

The contract operates under the assumption that liquidity is consistently provided to the pair between the contract's token and the native currency. However, there is a possibility that liquidity is provided to a different pair. This inadequacy in liquidity provision in the main pair could expose the contract to risks. Specifically, during eligible transactions, where the contract attempts to swap tokens with the main pair, a failure may occur if liquidity has been added to a pair other than the primary one. Consequently, transactions triggering the swap functionality will result in a revert.

Shell

```
pancakeRouter.swapExactTokensForETHSupportingFeeOnTransferT  
okens(tokenAmount, 0, path, address(this),  
block.timestamp);  
function _swapTokensForEth(uint256 tokenAmount) private {  
...  
pancakeRouter.addLiquidityETH{value:  
ethAmount}(address(this), tokenAmount, 0, 0, DEAD,  
block.timestamp);
```

## Recommendation

The team is advised to implement a runtime mechanism to check if the pair has adequate liquidity provisions. This feature allows the contract to omit token swaps if the pair does not have adequate liquidity provisions, significantly minimizing the risk of potential failures.

Furthermore, the team could ensure the contract has the capability to switch its active pair in case liquidity is added to another pair.

Additionally, the contract could be designed to tolerate potential reverts from the swap functionality, especially when it is a part of the main transfer flow. This can be achieved by executing the contract's token swaps in a non-reversible manner, thereby ensuring a more resilient and predictable operation.

## PTRP - Potential Transfer Revert Propagation

|                    |                                        |
|--------------------|----------------------------------------|
| <b>Criticality</b> | Minor / Informative                    |
| <b>Location</b>    | GravityProtocol_v4_post_audit.sol#L576 |
| <b>Status</b>      | Unresolved                             |

### Description

The contract sends funds to a `protocolFundWallet` as part of the transfer flow. This address can either be a wallet address or a contract. If the address belongs to a contract then it may revert from incoming payment. As a result, the error will propagate to the token's contract and revert the transfer. Additionally the contract may attempt to spend all the gas from the incoming payment not leaving enough for the rest of the operations. As a result the call will run out of gas and this will also propagate to the token's contract and revert the transfer.

```
Shell
uint256 protocolFundETH = address(this).balance -
initialBalance;
if (protocolFundETH > 0) {
    (bool success, ) = protocolFundWallet.call{value:
protocolFundETH}("");
    if (!success) {
    }
}
}
```

## Recommendation

The contract should tolerate the potential revert from the underlying contracts when the interaction is part of the main transfer flow. This could be achieved by not allowing set contract addresses or by sending the funds in a non-revertable way while also protecting the contract from a potential gas drain.



## SMS - Swap Mechanism Suppression

|                    |                                            |
|--------------------|--------------------------------------------|
| <b>Criticality</b> | Minor / Informative                        |
| <b>Location</b>    | GravityProtocol_v4_post_audit.sol#L519,532 |
| <b>Status</b>      | Unresolved                                 |

### Description

The contract implements a swap-and-liquify mechanism that depends on the relationship between `maxTxAmount` and `numTokensSellToAddToLiquidity`. However, due to the logic that caps the contract's token balance to `maxTxAmount` before checking whether the swap threshold is reached, the effective condition for triggering swaps becomes dependent on the owner-controlled parameters. If the owner sets `maxTxAmount` lower than `numTokensSellToAddToLiquidity`, the threshold condition can never be satisfied, even if the contract accumulates a large number of fee tokens. As a result, the swap-and-liquify process can be permanently disabled, causing tokens collected as fees to accumulate in the contract without being converted to liquidity or protocol funds, which alters the expected tokenomics and can lead to unpredictable future market behavior when swaps are re-enabled.

Shell

```
uint256 contractTokenBalance = balanceOf(address(this));
if (contractTokenBalance >= maxTxAmount) {
    contractTokenBalance = maxTxAmount;
}

bool overMinTokenBalance = contractTokenBalance >=
numTokensSellToAddToLiquidity;
if (
    overMinTokenBalance &&
    !_inSwapAndLiquify &&
    from != pancakePair &&
    swapAndLiquifyEnabled
) {
```

```
contractTokenBalance = numTokensSellToAddToLiquidity;  
_swapAndLiquify(contractTokenBalance);  
}
```

## Recommendation

The team is advised to ensure that the swap trigger condition is independent of the capped transaction amount or to enforce a constraint that guarantees `maxTxAmount` is always greater than or equal to `numTokensSellToAddToLiquidity`. This can be achieved by validating parameter updates or by restructuring the logic so that the threshold check uses the actual contract balance before applying any caps. This will help preserve the intended functionality of the swap-and-liquify mechanism and prevent the owner from unintentionally or deliberately disabling it.

## L13 - Divide before Multiply Operation

|                    |                                                    |
|--------------------|----------------------------------------------------|
| <b>Criticality</b> | Minor / Informative                                |
| <b>Location</b>    | GravityProtocol_v4_post_audit.sol#L556,568,618,624 |
| <b>Status</b>      | Unresolved                                         |

### Description

It is important to be aware of the order of operations when performing arithmetic calculations. This is especially important when working with large numbers, as the order of operations can affect the final result of the calculation. Performing divisions before multiplications may cause loss of precision.

```
Shell
uint256 half = liquidityTokens / 2;

uint256 liquidityETH = (newBalance * half) /
tokensToSwap;
```

### Recommendation

To avoid this issue, it is recommended to carefully consider the order of operations when performing arithmetic calculations in Solidity. It's generally a good idea to use parentheses to specify the order of operations. The basic rule is that the multiplications should be prior to the divisions.

## L16 - Validate Variable Setters

|                    |                                       |
|--------------------|---------------------------------------|
| <b>Criticality</b> | Minor / Informative                   |
| <b>Location</b>    | GravityProtocol_v4_post_audit.sol#L77 |
| <b>Status</b>      | Unresolved                            |

### Description

The contract performs operations on variables that have been configured on user-supplied input. These variables are missing of proper check for the case where a value is zero. This can lead to problems when the contract is executed, as certain actions may not be properly handled when the value is zero.

```
Shell
_owner = msgSender
```

### Recommendation

By adding the proper check, the contract will not allow the variables to be configured with zero value. This will ensure that the contract can handle all possible input values and avoid unexpected behavior or errors. Hence, it can help to prevent the contract from being exploited or operating unexpectedly.

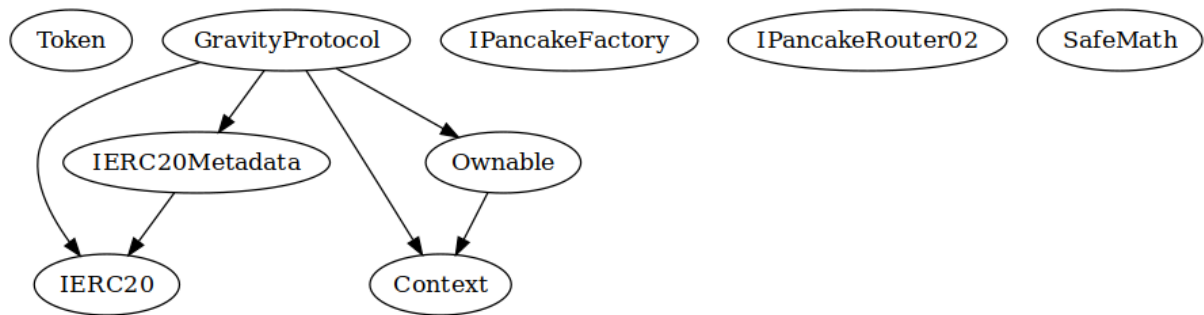
# Functions Analysis

| Contract        | Type                 | Bases                                                 |            |           |
|-----------------|----------------------|-------------------------------------------------------|------------|-----------|
|                 | Function Name        | Visibility                                            | Mutability | Modifiers |
|                 |                      |                                                       |            |           |
| GravityProtocol | Implementation       | Context,<br>IERC20,<br>IERC20Meta<br>data,<br>Ownable |            |           |
|                 |                      | Public                                                | ✓          | -         |
|                 | name                 | Public                                                |            | -         |
|                 | symbol               | Public                                                |            | -         |
|                 | decimals             | Public                                                |            | -         |
|                 | totalSupply          | Public                                                |            | -         |
|                 | balanceOf            | Public                                                |            | -         |
|                 | transfer             | Public                                                | ✓          | -         |
|                 | allowance            | Public                                                |            | -         |
|                 | approve              | Public                                                | ✓          | -         |
|                 | transferFrom         | Public                                                | ✓          | -         |
|                 | increaseAllowance    | Public                                                | ✓          | -         |
|                 | decreaseAllowance    | Public                                                | ✓          | -         |
|                 | tokenFromReflection  | Public                                                |            | -         |
|                 | reflectionFromToken  | Public                                                |            | -         |
|                 | totalFees            | Public                                                |            | -         |
|                 | isExcludedFromReward | Public                                                |            | -         |
|                 | isExcludedFromFee    | Public                                                |            | -         |
|                 | excludeFromReward    | Public                                                | ✓          | onlyOwner |

|  |                                  |          |   |             |
|--|----------------------------------|----------|---|-------------|
|  | includeInReward                  | External | ✓ | onlyOwner   |
|  | excludeFromFee                   | Public   | ✓ | onlyOwner   |
|  | includeInFee                     | Public   | ✓ | onlyOwner   |
|  | enableTrading                    | External | ✓ | onlyOwner   |
|  | setBlacklist                     | External | ✓ | onlyOwner   |
|  | setProtocolFundWallet            | External | ✓ | onlyOwner   |
|  | setMaxTxAmount                   | External | ✓ | onlyOwner   |
|  | setMaxWalletAmount               | External | ✓ | onlyOwner   |
|  | setSwapAndLiquifyEnabled         | Public   | ✓ | onlyOwner   |
|  | setCooldownEnabled               | External | ✓ | onlyOwner   |
|  | setCooldownTime                  | External | ✓ | onlyOwner   |
|  | setAntiBotBlocks                 | External | ✓ | onlyOwner   |
|  | setEarlySellerExtraFee           | External | ✓ | onlyOwner   |
|  | setNumTokensSellToAddToLiquidity | External | ✓ | onlyOwner   |
|  | _approve                         | Private  | ✓ |             |
|  | _transfer                        | Private  | ✓ |             |
|  | _swapAndLiquify                  | Private  | ✓ | lockTheSwap |
|  | _swapTokensForEth                | Private  | ✓ |             |
|  | _addLiquidity                    | Private  | ✓ |             |
|  | _tokenTransferSnipe              | Private  | ✓ |             |
|  | _tokenTransfer                   | Private  | ✓ |             |
|  | _transferStandard                | Private  | ✓ |             |
|  | _transferToExcluded              | Private  | ✓ |             |
|  | _transferFromExcluded            | Private  | ✓ |             |
|  | _transferBothExcluded            | Private  | ✓ |             |

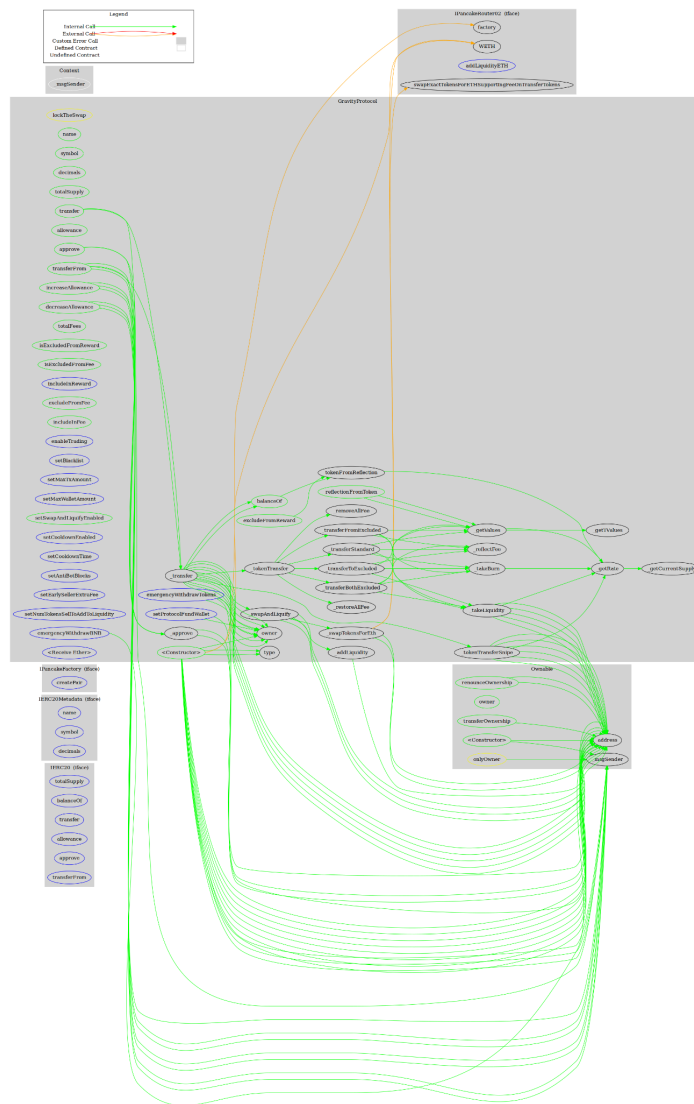
|  |                         |          |         |           |
|--|-------------------------|----------|---------|-----------|
|  | _getValues              | Private  |         |           |
|  | _getTValues             | Private  |         |           |
|  | _getRate                | Private  |         |           |
|  | _getCurrentSupply       | Private  |         |           |
|  | _takeLiquidity          | Private  | ✓       |           |
|  | _takeBurn               | Private  | ✓       |           |
|  | _reflectFee             | Private  | ✓       |           |
|  | _removeAllFee           | Private  | ✓       |           |
|  | _restoreAllFee          | Private  | ✓       |           |
|  | emergencyWithdrawBNB    | External | ✓       | onlyOwner |
|  | emergencyWithdrawTokens | External | ✓       | onlyOwner |
|  |                         | External | Payable | -         |

## Inheritance Graph





## Flow Graph



## Summary

Gravity Protocol contract implements a token mechanism. This audit investigates security issues, business logic concerns and potential improvements. There are some functions that can be abused by the owner like stop transactions, manipulate the fees and massively blacklist addresses. A multi-wallet signing pattern will provide security against potential hacks. Temporarily locking the contract or renouncing ownership will eliminate all the contract threats.

## Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security. Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

# About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.



A **TAC Security** Company

The Cyberscope team

[cyberscope.io](https://cyberscope.io)